

ПРАКТИЧЕСКАЯ РАБОТА №3. Установка окружения и первый запуск

Цель

Настроить рабочее место в Windows, создать каркас Django-проекта в VS Code, подключить PostgreSQL и Redis.

Установить Redis (Memurai)

Официальный Redis на Windows не работает. Используем **Memurai** — совместимый, работает как служба.

Открыть <https://www.memurai.com/get-memurai>

Скачать **Memurai Developer** (бесплатно).

Запустить `.msi`.

Все настройки по умолчанию.

Port: `6379`.

Создать папку проекта

Открыть **Проводник** Windows.

Создать папку `C:\study\ams` (используйте ваше название)

Открыть её в VS Code: **File** → **Open Folder** → `C:\study\ams`.

Создать виртуальное окружение

```
python -m venv .venv
```

Дождаться окончания. В папке появится `.venv/`.

Активировать:

```
.venv\Scripts\activate
```

Признак активации — `(.venv)` в начале строки:

```
(.venv) C:\study\ams>
```

```
Problems Output Debug Console Terminal Ports DBCode
Windows PowerShell
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

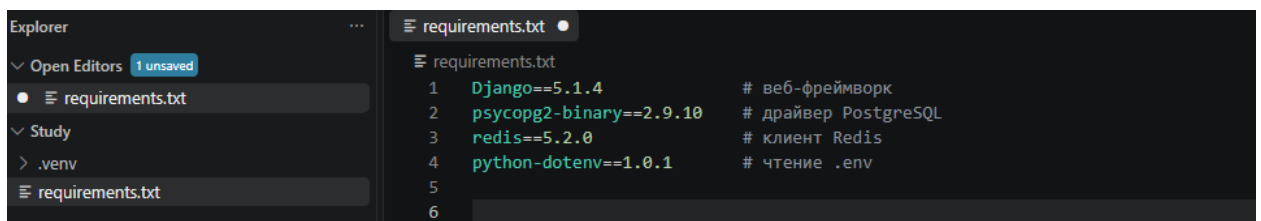
PS C:\Study> dir
PS C:\Study> python -m venv .venv
PS C:\Study> c:\Study\.venv\Scripts\activate
PS C:\Study> .venv\Scripts\activate
(.venv) PS C:\Study> 
```

Все дальнейшие команды — только в активированном `venv`.

Создать `requirements.txt`

В VS Code в панели Explorer (слева) нажать **New File** → имя `requirements.txt`.

```
Django==5.1.4           # веб-фреймворк
psycopg2-binary==2.9.10 # драйвер PostgreSQL
redis==5.2.0            # клиент Redis
python-dotenv==1.0.1    # чтение .env
```

The screenshot shows the VS Code interface. On the left, the Explorer panel shows a file named 'requirements.txt' under the 'Study' folder. On the right, the Editor panel shows the content of 'requirements.txt' with line numbers 1 through 6. The content matches the text in the previous block.

```
Explorer
Open Editors 1 unsaved
requirements.txt
Study
.venv
requirements.txt

requirements.txt
1 Django==5.1.4           # веб-фреймворк
2 psycopg2-binary==2.9.10 # драйвер PostgreSQL
3 redis==5.2.0            # клиент Redis
4 python-dotenv==1.0.1    # чтение .env
5
6
```

Установить:

`pip install -r requirements.txt`

```
Problems Output Debug Console Terminal Ports DBCode
(C) Корпорация Майкрософт (Microsoft Corporation). Все права защищены.

PS C:\Study> dir
PS C:\Study> python -m venv .venv
PS C:\Study> c:\Study\.venv\Scripts\activate
PS C:\Study> .venv\Scripts\activate
(.venv) PS C:\Study> pip install -r requirements.txt

[notice] A new release of pip is available: 25.1.1 -> 26.2.1
[notice] To update, run: python.exe -m pip install --upgrade pip
(.venv) PS C:\Study> python.exe -m pip install --upgrade pip
Requirement already satisfied: pip in c:\study\.venv\lib\site-packages (25.1.1)
```

```
(.venv) PS C:\Study> pip install django
Collecting django
  Downloading django-6.1.1-py3-none-any.whl.metadata (3.9 kB)
Collecting asgiref>=3.9.1 (from django)
  Downloading asgiref-3.12.1-py3-none-any.whl.metadata (9.4 kB)
Collecting sqlparse>=0.5.0 (from django)
  Downloading sqlparse-0.6.0-py3-none-any.whl.metadata (6.0 kB)
```

```
----- 8.4/8.4 MB 1.0 MB/s 0:00:08
Downloading asgiref-3.12.1-py3-none-any.whl (25 kB)
Using cached tzdata-2026.4-py2.py3-none-any.whl (347 kB)
Installing collected packages: tzdata, sqlparse, asgiref, django
Successfully installed asgiref-3.12.1 django-6.1.1 sqlparse-0.6.0 tzdata-2026.4
(.venv) PS C:\Study> pip install psycopg2-binary
Collecting psycopg2-binary
  Downloading psycopg2_binary-2.9.13-cp313-cp313-win_amd64.whl (2.8 MB)
----- 2.8/2.8 MB 341.3 kB/s 0:00:08
Installing collected packages: psycopg2-binary
Successfully installed psycopg2-binary-2.9.13
(.venv) PS C:\Study> pip install redis
Collecting redis
  Using cached redis-8.1.0-py3-none-any.whl.metadata (16 kB)
Using cached redis-8.1.0-py3-none-any.whl (560 kB)
Installing collected packages: redis
Successfully installed redis-8.1.0
(.venv) PS C:\Study> pip install python-dotenv
Collecting python-dotenv
  Using cached python_dotenv-1.2.3-py3-none-any.whl.metadata (29 kB)
Using cached python_dotenv-1.2.3-py3-none-any.whl (22 kB)
Installing collected packages: python-dotenv
```

Создать Django-проект

```
(.venv) PS C:\Study> django-admin startproject config .
(.venv) PS C:\Study> 
```

Проверить структуру (dir):

Mode	LastWriteTime	Length	Name
d-----	16.09.2026 11:31		.venv
d-----	16.09.2026 11:35		config
-a----	16.09.2026 11:35	684	manage.py
-a----	16.09.2026 11:32	0	requirements.txt

Создать базу данных

Открыть **новый** cmd (не тот, где venv):

```
$env:Path += ";C:\Program Files\PostgreSQL\18\bin"
```

```
>> psql -version
```

```
psql -U postgres
```

```
(.venv) PS C:\Study> $env:Path += ";C:\Program Files\PostgreSQL\18\bin"
>> psql --version
psql (PostgreSQL) 18.6
(.venv) PS C:\Study> psql -U postgres
```

Ввести пароль, который задали при установке PostgreSQL.

В приглашении postgres=#:

```
postgres=# CREATE USER ams WITH PASSWORD 'ams';
CREATE ROLE
postgres=# CREATE DATABASE ams OWNER ams;
CREATE DATABASE
postgres=# GRANT ALL ON SCHEMA public TO ams;
GRANT
postgres=# \q
(.venv) PS C:\Study> psql -U postgres
Password for user postgres:
```

Проверить подключение:

```
(.venv) PS C:\Study> psql -U ams -d ams -c "SELECT version();"
Password for user ams:
          version
-----
PostgreSQL 18.6 on x86_64-windows, compiled by msvc-19.44.35228, 64-bit
(1 row)
```

Создать .env.example

```
requirements.txt ● .env.example ●
.env.example
1 SECRET_KEY=dev-secret-change-me
2 DEBUG=True
3 ALLOWED_HOSTS=localhost,127.0.0.1
4
5 POSTGRES_DB=ams
6 POSTGRES_USER=ams
7 POSTGRES_PASSWORD=ams
8 POSTGRES_HOST=localhost
9 POSTGRES_PORT=5432
10
11 REDIS_URL=redis://localhost:6379/1
```

Скопировать в .env:

```
(.venv) PS C:\Study> copy .env.example .env
(.venv) PS C:\Study> 
```

Сгенерировать SECRET_KEY:

```
Problems Output Debug Console Terminal Ports DBCode
-----
PostgreSQL 18.6 on x86_64-windows, compiled by msvc-19.44.35228, 64-bit
(1 row)

(.venv) PS C:\Study> copy .env.example .env
(.venv) PS C:\Study> python -c "from django.core.management.utils import get_random_secret_key as k; print(k())"
&txef*11y#yfg%qi!-m=qt$h33u1!b-92*j3i%q&4it(*8s4+v
(.venv) PS C:\Study> 
```

Скопировать вывод. Открыть .env в VS Code, заменить строку SECRET_KEY=... на:

```
settings.py 7 • .env X
.env
1
2 SECRET_KEY=&txef*11y#yfg%qi!-m=qt$h33u1!b-92*j3i%q&4it(*8s4+v
3
4 DEBUG=True
5 ALLOWED_HOSTS=localhost,127.0.0.1
6
7 POSTGRES_DB=ams
8 POSTGRES_USER=ams
9 POSTGRES_PASSWORD=ams
10 POSTGRES_HOST=localhost
11 POSTGRES_PORT=5432
12
13 REDIS_URL=redis://localhost:6379/1
```

Создать `.gitignore`

```
requirements.txt • .env.example • .env X • .gitignore X
.gitignore
1 # Python
2 __pycache__/
3 *.py[cod]
4 .venv/
5 venv/
6
7 # Django
8 *.log
9 db.sqlite3
10 staticfiles/
11 media/
12
13 # Окружение
14 .env
15
16 # IDE
17 .vscode/
18 .idea/
```

Настроить `config/settings.py`

Открыть `config/settings.py` в VS Code. Полностью заменить содержимое:

settings.py X

config > settings.py > ...

```
1  import os                                # работа с переменными окружения
2  from pathlib import Path                  # удобные пути
3
4  from dotenv import load_dotenv           # библиотека для чтения .env
5
6  BASE_DIR = Path(__file__).resolve().parent.parent # корень проекта (две папки вверх)
7
8  load_dotenv(BASE_DIR / ".env")           # читаем .env и кладем переменные
9
10 SECRET_KEY = os.getenv("SECRET_KEY")     # ключ для подписи сессий и CSRF
11 DEBUG = os.getenv("DEBUG", "True") == "True" # True – режим разработки
12 ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "localhost").split(",") # список разрешенных
13
14 INSTALLED_APPS = [                       # список подключённых приложений
15     "django.contrib.admin",              # встроенная админка
16     "django.contrib.auth",               # встроенная авторизация
17     "django.contrib.contenttypes",       # типы контента (нужно для auth)
18     "django.contrib.sessions",           # сессии
19     "django.contrib.messages",           # всплывающие сообщения
20     "django.contrib.staticfiles",        # статика (css/js)
21 ]
22
23 MIDDLEWARE = [                           # промежуточные слои между запросом и view
```

```
24     "django.middleware.security.SecurityMiddleware", # базовая безопасн
25     "django.contrib.sessions.middleware.SessionMiddleware", # сессии
26     "django.middleware.common.CommonMiddleware", # общие правила
27     "django.middleware.csrf.CsrfViewMiddleware", # защита от CSRF
28     "django.contrib.auth.middleware.AuthenticationMiddleware", # request.user
29     "django.contrib.messages.middleware.MessageMiddleware", # messages fram
30 ]
31
32 ROOT_URLCONF = "config.urls"             # где лежит корневой urls.py
33
34 TEMPLATES = [                             # настройки шаблонов
35     {
36         "BACKEND": "django.template.backends.django.DjangoTemplates", # движок Django
37         "DIRS": [BASE_DIR / "templates"], # где искать общие шаблоны
38         "APP_DIRS": True, # искать также в templates/ приложений
39         "OPTIONS": {
40             "context_processors": [ # переменные во всех шаблонах
41                 "django.template.context_processors.request", # request
42                 "django.contrib.auth.context_processors.auth", # user
43                 "django.contrib.messages.context_processors.messages", # messages
44             ],
45         }
```

```

config > settings.py > ...
43         "django.contrib.messages.context_processors.messages", # messages
44     ],
45 },
46 },
47 ]
48
49 WSGI_APPLICATION = "config.wsgi.application" # точка входа для сервера
50
51 DATABASES = { # настройки БД
52     "default": {
53         "ENGINE": "django.db.backends.postgresql", # драйвер PostgreSQL
54         "NAME": os.getenv("POSTGRES_DB"), # имя БД
55         "USER": os.getenv("POSTGRES_USER"), # пользователь
56         "PASSWORD": os.getenv("POSTGRES_PASSWORD"), # пароль
57         "HOST": os.getenv("POSTGRES_HOST"), # хост
58         "PORT": os.getenv("POSTGRES_PORT"), # порт
59     }
60 }
61
62 CACHES = { # кэш через Redis
63     "default": {
64         "BACKEND": "django.core.cache.backends.redis.RedisCache", # встроенный Redis-бэкенд
65         "LOCATION": os.getenv("REDIS_URL"), # адрес Redis
66     }

```

```

62 CACHES = { # кэш через Redis
63     "default": {
64         "BACKEND": "django.core.cache.backends.redis.RedisCache", # встроенный Redis-бэкенд
65         "LOCATION": os.getenv("REDIS_URL"), # адрес Redis
66     }
67 }
68
69 SESSION_ENGINE = "django.contrib.sessions.backends.cache" # сессии — в Redis
70 SESSION_CACHE_ALIAS = "default" # какой кэш использовать
71
72 LANGUAGE_CODE = "ru-ru" # русский язык
73 TIME_ZONE = "Europe/Moscow" # московское время
74 USE_I18N = True # включить переводы
75 USE_TZ = True # хранить время с таймзоной
76
77 STATIC_URL = "static/" # URL для статики
78 STATICFILES_DIRS = [BASE_DIR / "static"] # где искать статику
79 MEDIA_URL = "media/" # URL для загруженных файлов
80 MEDIA_ROOT = BASE_DIR / "media" # папка для файлов
81
82 DEFAULT_AUTO_FIELD = "django.db.models.BigAutoField" # тип PK по умолчанию

```

Создать папки

В терминале VS Code:

```
(.venv) PS C:\Study> python manage.py migrate
Operations to perform:
  Apply all migrations: admin, auth, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying auth.0001_initial... OK
  Applying admin.0001_initial... OK
  Applying admin.0002_logentry_remove_auto_add... OK
```

Запустить сервер

В VS Code в панели **Run and Debug** (Ctrl+Shift+D) создать `launch.json` для отладки или запустить из терминала:

```
Applying sessions.0001_initial... OK
(.venv) PS C:\Study> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
September 16, 2026 - 12:18:20
```

```
Django version 6.1.1, using settings 'config.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a p
roduction WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/ho
deployment/
[16/Sep/2026 12:19:24] "GET / HTTP/1.1" 200 12532
```



Установка прошла успешно! Поздравляем!

Посмотреть [примечания к выпуску](#) для Django 6.1

Вы видите данную страницу, потому что указали `DEBUG=True` в файле настроек и не настроили ни одного обработчика URL-адресов.

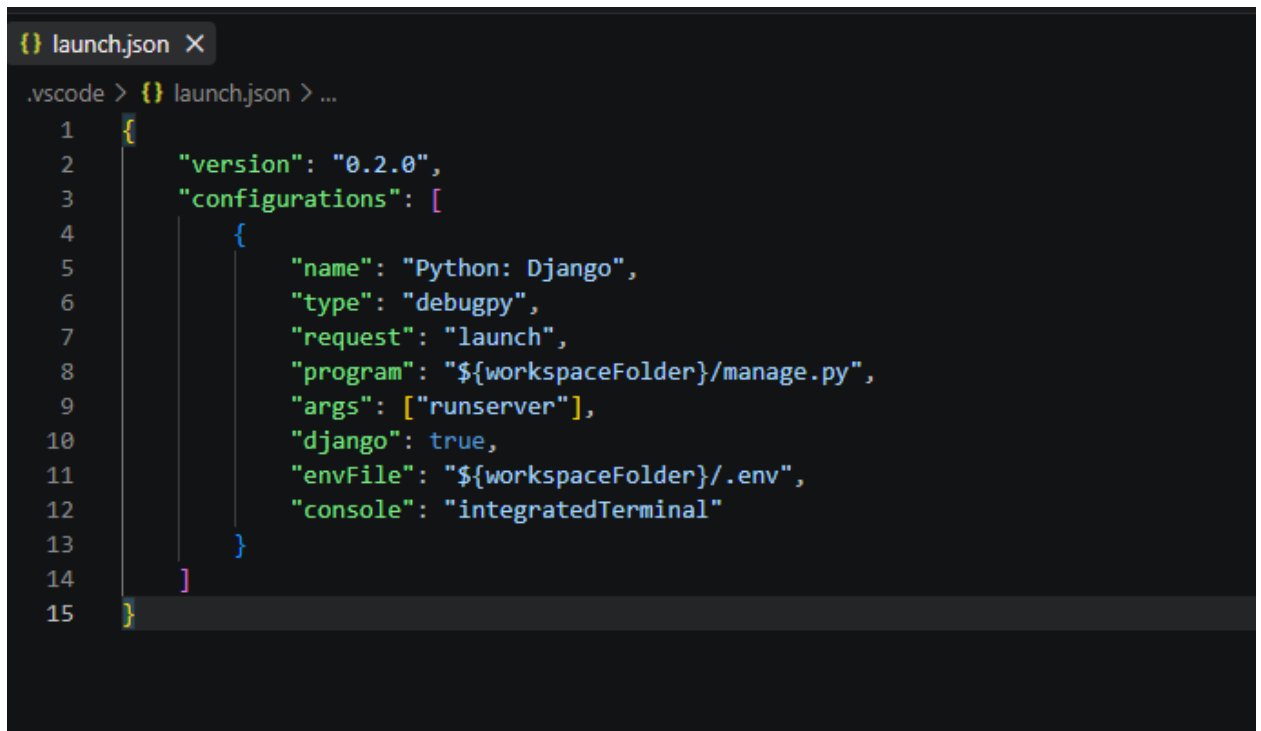
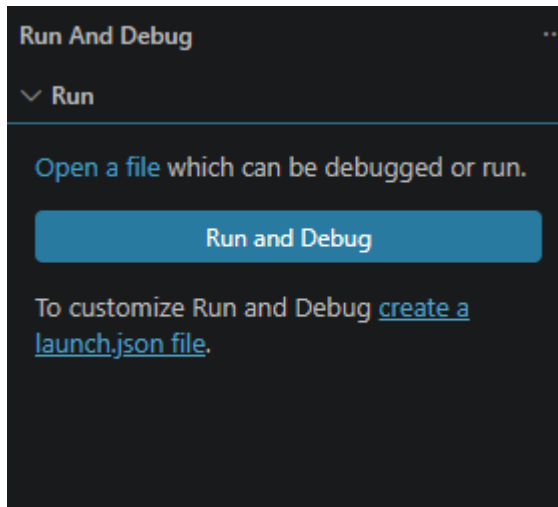
django

Остановить сервер: `Ctrl+C` в терминале.

Настроить VS Code для отладки (опционально)

В VS Code: **Run and Debug** → create a **launch.json** file → Python → Django.

launch.json должен получиться примерно таким:



Теперь можно запускать сервер кнопкой **F5** и ставить breakpoints.

Инициализировать Git

```
(.venv) PS C:\Study> git init
>> git commit -m "ПР 1: каркас проекта"
Reinitialized existing Git repository in C:/Study/.git/
On branch main
Your branch is up to date with 'origin/main'.
nothing to commit, working tree clean
>> git branch -M main
>> git push -u origin main
```

Создать справочники (Location, Category, Vendor, Model) и кастомного пользователя, зарегистрировать в админке.

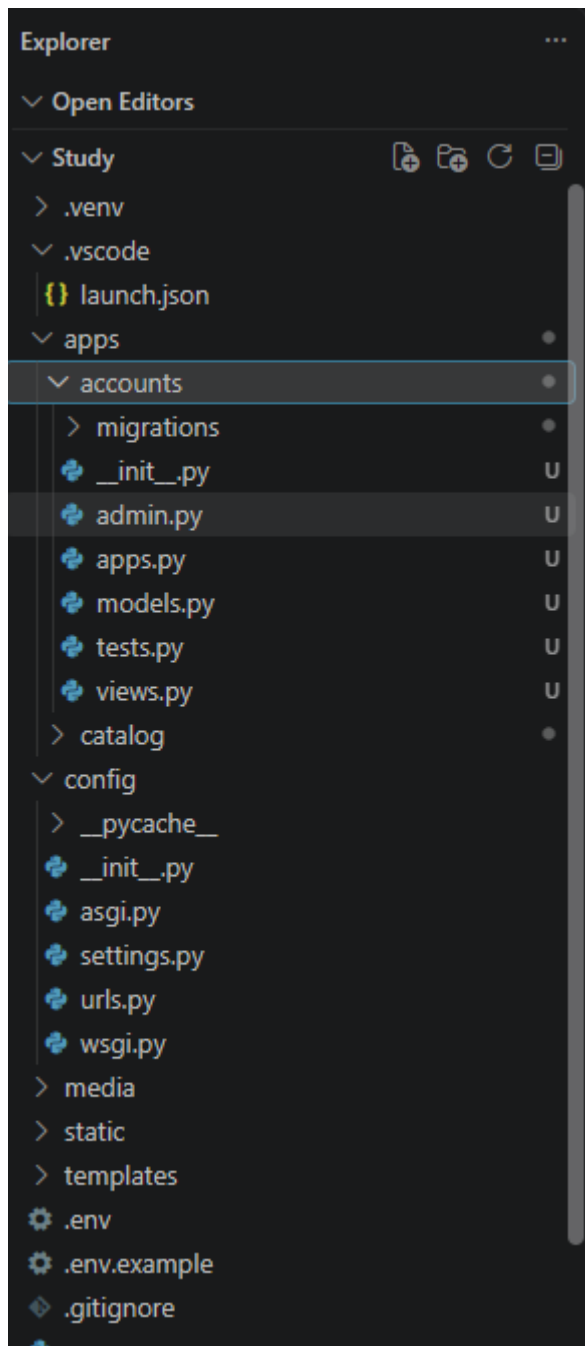
Создать приложения

В терминале VS Code (venv активирован):

```
(.venv) PS C:\Study> mkdir apps
>> python manage.py startapp accounts apps/accounts
>> python manage.py startapp catalog apps/catalog
```

Каталог: C:\Study

Mode	LastWriteTime	Length	Name
------	---------------	--------	------



Исправить `apps.py` приложений

`apps/accounts/apps.py`

```
admin.py U X  apps.py ...\accounts U X  apps.py ...\catalog U X
apps > accounts > apps.py > AccountsConfig
1  from django.apps import AppConfig
2
3
4  class AccountsConfig(AppConfig):
5      default_auto_field = "django.db.models.BigAutoField"
6      name = "apps.accounts"
7      verbose_name = "Пользователи"
```

apps/catalog/apps.py

```
admin.py U X  apps.py ...\accounts U X  apps.py ...\catalog U X
apps > catalog > apps.py > CatalogConfig
1  from django.apps import AppConfig
2
3
4  class CatalogConfig(AppConfig):
5      default_auto_field = "django.db.models.BigAutoField"
6      name = "apps.catalog"
7      verbose_name = "Справочники"
```

Добавить в INSTALLED_APPS

Открыть config/settings.py, найти INSTALLED_APPS, добавить **в конец** (после "django.contrib.staticfiles"):

```
apps.py U X settings.py M X
config > settings.py > ...
9
10 SECRET_KEY = os.getenv("SECRET_KEY") # ключ для подписи сессий и CSRF
11 DEBUG = os.getenv("DEBUG", "True") == "True" # True – режим разработки
12 ALLOWED_HOSTS = os.getenv("ALLOWED_HOSTS", "localhost").split(",") # список разр
13
14 INSTALLED_APPS = [] # список подключённых приложений
15     "django.contrib.admin", # встроенная админка
16     "django.contrib.auth", # встроенная авторизация
17     "django.contrib.contenttypes", # типы контента (нужно для auth)
18     "django.contrib.sessions", # сессии
19     "django.contrib.messages", # всплывающие сообщения
20     "django.contrib.staticfiles", # статика (css/js)
21     "apps.accounts", # наше: пользователи
22     "apps.catalog", # наше: справочники
23
```

Модель пользователя

`apps/accounts/models.py` — заменить содержимое:

```
models.py U X
apps > accounts > models.py > ...
1 from django.contrib.auth.models import AbstractUser # базовая модель пользователя
2 from django.db import models # поля моделей
3
4
5 class User(AbstractUser): # наследуемся от AbstractUser
6     class Role(models.TextChoices): # перечисление ролей
7         ADMIN = "admin", "Администратор" # код, отображаемое имя
8         KEEPER = "keeper", "Кладовщик"
9         MOL = "mol", "МОЛ"
10        AUDITOR = "auditor", "Аудитор"
11
12        full_name = models.CharField("ФИО", max_length=255, blank=True) # ФИО, можно пусто
13        role = models.CharField( # роль
14            "Роль", max_length=32,
15            choices=Role.choices, # ограничить значениями Role
16            default=Role.MOL, # по умолчанию МОЛ
17        )
18
19        def __str__(self): # как объект печатается
20            return self.full_name or self.username # ФИО, если пусто – логин
```

Указать `AUTH_USER_MODEL`

В `config/settings.py` добавить **после** `WSGI_APPLICATION`:

```
models.py U X settings.py M
config > settings.py > ...
40 APP_DIRS = True, # искать также в templates/ прил
41 "OPTIONS": {
42     "context_processors": [ # переменные во всех шаблонах
43         "django.template.context_processors.request", # request
44         "django.contrib.auth.context_processors.auth", # user
45         "django.contrib.messages.context_processors.messages", # messages
46     ],
47 },
48 },
49 ]
50
51 WSGI_APPLICATION = "config.wsgi.application" # точка входа для сервера
52 AUTH_USER_MODEL = "accounts.User" # наша модель пользователя
53
54 DATABASES = { # настройки БД
55     "default": {
56         "ENGINE": "django.db.backends.postgresql", # драйвер PostgreSQL
57         "NAME": os.getenv("POSTGRES_DB"), # имя БД
58         "USER": os.getenv("POSTGRES_USER"), # пользователь
59         "PASSWORD": os.getenv("POSTGRES_PASSWORD"), # пароль
60         "HOST": os.getenv("POSTGRES_HOST"), # хост
61         "PORT": os.getenv("POSTGRES_PORT"), # порт
62     }
63 }
```

```
TABASE ams;"
Password for user postgres:
DROP DATABASE
(.venv) PS C:\Study> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -c "CREATE
DATABASE ams OWNER ams;"
Password for user postgres:
CREATE DATABASE
(.venv) PS C:\Study> & "C:\Program Files\PostgreSQL\16\bin\psql.exe" -U postgres -d ams -c "
GRANT ALL ON SCHEMA public TO ams;"
Password for user postgres:
GRANT
(.venv) PS C:\Study> 
```

Модели справочников

apps/catalog/models.py — заменить содержимое:

```

1  from django.db import models                # поля моделей
2
3
4  class Location(models.Model):                # модель «Локация»
5      name = models.CharField("Название", max_length=255) # название, до 255 симв.
6
7      class Meta:                             # мета-настройки
8          verbose_name = "Локация"             # имя в ед. числе
9          verbose_name_plural = "Локации"      # имя во мн. числе
10
11     def __str__(self):                       # как печатается объект
12         return self.name                     # возвращаем название
13
14
15 class Category(models.Model):                # модель «Категория»
16     name = models.CharField("Название", max_length=255)
17
18     class Meta:
19         verbose_name = "Категория"
20         verbose_name_plural = "Категории"
21
22     def __str__(self):
23         return self.name
24
25
26 class Vendor(models.Model):                  # модель «Производитель»
27     name = models.CharField("Название", max_length=255, unique=True) # уникальное имя
28
29     class Meta:
30         verbose_name = "Производитель"
31         verbose_name_plural = "Производители"

```

```

29     class Meta:
30         verbose_name = "Производитель"
31         verbose_name_plural = "Производители"
32
33     def __str__(self):
34         return self.name
35
36
37 class Model(models.Model):                   # модель «Модель оборудования»
38     vendor = models.ForeignKey(               # связь с производителем
39         Vendor, on_delete=models.PROTECT,    # запрет удаления, если есть ссылки
40         verbose_name="Производитель",
41     )
42     category = models.ForeignKey(             # связь с категорией
43         Category, on_delete=models.PROTECT,
44         verbose_name="Категория",
45     )
46     name = models.CharField("Модель", max_length=255) # название модели
47
48     class Meta:
49         verbose_name = "Модель"
50         verbose_name_plural = "Модели"
51
52     def __str__(self):
53         return f"{self.vendor.name} {self.name}" # например, «HP ProBook 450»

```

Админка

apps/accounts/admin.py — заменить содержимое:

```
admin.py U X
apps > accounts > admin.py > CustomUserAdmin
1 from django.contrib import admin # админка
2 from django.contrib.auth.admin import UserAdmin # базовая админка User
3
4 from .models import User # наша модель
5
6
7 @admin.register(User) # регистрируем User
8 class CustomUserAdmin(UserAdmin): # наследуемся от UserAdmin
9     list_display = ("username", "full_name", "role", "is_active") # колонки
10    list_filter = ("role", "is_active") # фильтры справа
11    fieldsets = UserAdmin.fieldsets + ( # стандартные секции + наша
12        ("Дополнительно", {"fields": ("full_name", "role")}),
13    )
```

```
admin.py U X
apps > accounts > admin.py > ...
1 from django.contrib import admin # админка
2
3 from .models import Category, Location, Model, Vendor # модели
4
5
6 @admin.register(Location) # регистрируем Location
7 class LocationAdmin(admin.ModelAdmin): # свой класс админки
8     list_display = ("id", "name") # колонки
9     search_fields = ("name",) # поиск по name
10
11 (class) Category
12 @admin.register(Category)
13 class CategoryAdmin(admin.ModelAdmin):
14     list_display = ("id", "name")
15     search_fields = ("name",)
16
17
18 @admin.register(Vendor)
19 class VendorAdmin(admin.ModelAdmin):
20     list_display = ("id", "name")
21     search_fields = ("name",)
22
23
24 @admin.register(Model)
25 class ModelAdmin(admin.ModelAdmin):
26     list_display = ("id", "name", "vendor", "category") # 4 колонки
27     list_filter = ("vendor", "category") # фильтры справа
28     search_fields = ("name",) # поиск
29     autocomplete_fields = ("vendor", "category") # выпадающий поиск
```

Создать миграции

```
System check identified no issues (0 silenced).
(.venv) PS C:\Study> python manage.py makemigrations
Migrations for 'catalog':
  apps\catalog\migrations\0001_initial.py
    + Create model Category
    + Create model Location
    + Create model Vendor
    + Create model Model
Migrations for 'accounts':
  apps\accounts\migrations\0001_initial.py
    + Create model User
(.venv) PS C:\Study>
```

Перед миграцией проверить файлы:

apps/accounts/admin.py

```
1  from django.contrib import admin
2  from django.contrib.auth.admin import UserAdmin
3
4  from .models import User
5
6
7  @admin.register(User)
8  class CustomUserAdmin(UserAdmin):
9      """Расширяем стандартную админку User.
10
11      list_display – колонки в списке.
12      list_filter – фильтры справа.
13      fieldsets – секции в форме редактирования.
14      """
15      list_display = ("username", "full_name", "role", "is_active")
16      list_filter = ("role", "is_active")
17      fieldsets = UserAdmin.fieldsets + (
18          ("Дополнительно", {"fields": ("full_name", "role")}),
19      )
```

apps/catalog/admin.py

```

apps > catalog > admin.py > ModelAdmin
1  from django.contrib import admin
2
3  from .models import Category, Location, Model, Vendor
4
5
6  @admin.register(Location)
7  class LocationAdmin(admin.ModelAdmin):
8      list_display = ("id", "name")
9      search_fields = ("name",)
10
11
12  @admin.register(Category)
13  class CategoryAdmin(admin.ModelAdmin):
14      list_display = ("id", "name")
15      search_fields = ("name",)
16
17
18  @admin.register(Vendor)
19  class VendorAdmin(admin.ModelAdmin):
20      list_display = ("id", "name")
21      search_fields = ("name",)
22
23
24  @admin.register(Model)
25  class ModelAdmin(admin.ModelAdmin):
26      list_display = ("id", "name", "vendor", "category")
27      list_filter = ("vendor", "category")
28      search_fields = ("name",)
29      autocomplete_fields = ("vendor", "category")

```

apps/catalog/models.py

```
1  from django.db import models
2
3
4  class Location(models.Model):
5      name = models.CharField("Название", max_length=255)
6
7      class Meta:
8          verbose_name = "Локация"
9          verbose_name_plural = "Локации"
10
11     def __str__(self):
12         return self.name
13
14
15     class Category(models.Model):
16         name = models.CharField("Название", max_length=255)
17
18         class Meta:
19             verbose_name = "Категория"
20             verbose_name_plural = "Категории"
21
22         def __str__(self):
23             return self.name
24
25
26     class Vendor(models.Model):
27         name = models.CharField("Название", max_length=255, unique=True)
28
29         class Meta:
30             verbose_name = "Производитель"
```

```

29     class Meta:
30         verbose_name = "Производитель"
31         verbose_name_plural = "Производители"
32
33     def __str__(self):
34         return self.name
35
36
37     class Model(models.Model):
38         vendor = models.ForeignKey(
39             Vendor, on_delete=models.PROTECT,
40             verbose_name="Производитель",
41         )
42         category = models.ForeignKey(
43             Category, on_delete=models.PROTECT,
44             verbose_name="Категория",
45         )
46         name = models.CharField("Модель", max_length=255)
47
48         class Meta:
49             verbose_name = "Модель"
50             verbose_name_plural = "Модели"
51
52         def __str__(self):
53             return f"{self.vendor.name} {self.name}"

```

apps/accounts/models.py

```

apps > accounts > models.py > User > __str__
1  from django.contrib.auth.models import AbstractUser
2  from django.db import models
3
4
5  class User(AbstractUser):
6      class Role(models.TextChoices):
7          ADMIN = "admin", "Администратор"
8          KEEPER = "keeper", "Кладовщик"
9          MOL = "mol", "МОЛ"
10         AUDITOR = "auditor", "Аудитор"
11
12         full_name = models.CharField("ФИО", max_length=255, blank=True)
13         role = models.CharField(
14             "Роль", max_length=32,
15             choices=Role.choices,
16             default=Role.MOL,
17         )
18
19         def __str__(self):
20             return self.full_name or self.username

```

Миграции

```

(.venv) PS C:\Study> python manage.py makemigrations
No changes detected
(.venv) PS C:\Study> python manage.py migrate
Operations to perform:
  Apply all migrations: accounts, admin, auth, catalog, contenttypes, sessions
Running migrations:
  Applying contenttypes.0001_initial... OK
  Applying contenttypes.0002_remove_content_type_name... OK
  Applying auth.0001_initial... OK
  Applying auth.0002_alter_permission_name_max_length... OK
  Applying auth.0004_alter_user_username_opts... OK
  Applying auth.0005_alter_user_last_login_null... OK
  Applying auth.0006_require_contenttypes_0002... OK
  Applying auth.0007_alter_validators_add_error_messages... OK
  Applying auth.0008_alter_user_username_max_length... OK

```

Суперпользователь

```
(.venv) PS C:\Study> python manage.py createsuperuser
Имя пользователя: admin
Адрес электронной почты: admin@example.com
Password:
Password (again):
Superuser created successfully.
(.venv) PS C:\Study> 
```

Запустить и заполнить данными

```
(.venv) PS C:\Study> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).
September 16, 2026 - 13:36:00
Django version 6.1.1, using settings 'config.settings'
Starting WSGI development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.

WARNING: This is a development server. Do not use it in a production setting. Use a p
roduction WSGI or ASGI server instead.
For more information on production servers see: https://docs.djangoproject.com/en/6.1/howto/
```

